

Proposal — Entity Relationship Layer (“Knowledge Graph Lite”) for Seekra

Status: For architect review → implementation brief **Date:** 2026-08-17 **Author:** Platform review (post Brief #15 verification) **Scope discipline:** This proposal deliberately avoids a graph database and LLM-based relationship extraction. It builds on verified, already-deployed components.

1. Objective

Upgrade Seekra from “document-centric RAG” to “entity-centric intelligence”: let users and auditors ask *relationship* questions across the corpus —

- “Show every document connected to person X.”
- “Which contracts reference Article 5, and where is Article 5 defined?”
- “Which entities co-occur with company Y across all documents?”

These queries are impossible for plain vector RAG and are a direct differentiator for Seekra’s compliance/audit positioning.

2. Current State (verified, deployed)

Capability	Where	Status
Entity extraction at indexing (EN+AR regex)	entity_extraction.py, chunk_entities table	Live (Brief #13)
Arabic NER (persons/orgs/locations, exact offsets)	arabic_ner.py (Stanza)	Live (Brief #15)
Cross-reference resolution (mentioned ref → defining chunk)	cross_reference.py	Live (Brief #13.2)
Document version lineage	version_group_id, content_hash	Live (Brief #10)
PII-aware entity masking + lineage	pii_detection.py, audit chain	Live (Step 29/38, Brief #7)

Gap: entities are stored but *not linked*. There is no queryable relationship structure — only per-chunk rows and query-time regex resolution.

3. Recommendation: Postgres-native, two phases

No Neo4j / no new database. At Seekra’s current corpus scale, PostgreSQL handles graph-style queries via a plain edge table + recursive CTEs. (Apache AGE remains a *later* option if Cypher-style querying is ever needed; Neo4j only at multi-million-node scale.)

Phase 1 — Relationship capture + exploration API

1. **Edge table** entity_relationships: id, doc_id, chunk_id, subject_type, subject_value, relation, object_type, object_value, confidence, created_at
2. **Relationship inference at indexing — deterministic only (no LLM):**

- **DEFINES / REFERENCES** — from the existing cross-reference resolver (brief #13.2 logic, persisted instead of resolved at query time)
- **CO_OCCURS** — entities sharing a chunk (weight = chunk count)
- **MENTIONS** — entity → document
- **PART_OF_VERSION_CHAIN** — from `version_group_id`

3. **API:**

- `GET /api/entities/search?q=` — entity autocomplete (EN+AR)
- `GET /api/entities/{id}/graph?depth=1..2` — nodes + edges (recursive CTE, depth-capped, result-capped)
- `GET /api/documents/{id}/entities` — per-document entity map

4. **UI (bilingual EN+AR, RTL-safe — mandatory mirroring):**

- Entity panel on the document page (clickable chips)
- Simple graph view on the Provenance page (depth-1 radial list is acceptable for v1; no fancy physics library required)

5. **Audit:** ENTITY_GRAPH_QUERIED chained audit events (Brief #6 chain).

Phase 2 — Graph-enriched answers (chat integration)

- When a chat question names a known entity, inject its depth-1 neighborhood (documents, definitions, related entities) as additional context — *in addition to* vector retrieval, not replacing it.
- Reuses the Brief #12 structured prompt: graph facts arrive as an extra numbered block, cited the same way.
- Guard: PII masking rules (Step 29) apply to graph-derived context exactly as they do to retrieved chunks.

Explicitly out of scope (v1)

- LLM-based relationship extraction (cost/latency per chunk; revisit if deterministic relations prove insufficient)
- Neo4j / graph DB, embeddings for relations, ontology/schema management
- Depth > 2 traversal

4. Acceptance Criteria

1. Re-indexing an existing corpus populates `entity_relationships` with zero LLM calls (verified via worker logs).
2. `GET /api/entities/{id}/graph?depth=2` on a seeded test corpus returns expected nodes/edges; response < 500 ms at 100k edges.
3. UI entity panel renders identically in EN and AR (RTL verified), and clicking an entity navigates to its graph view with a working Back path (per provenance UX standards from Brief #8.2).
4. Chat answer for “who is connected to ?” cites both vector sources and graph facts, with PII masking intact in logs and responses.
5. Migration is additive and reversible; no changes to existing tables except new indexes.

5. Risks & Mitigations

Risk	Mitigation
Entity name collisions (same person string, different people)	v1 treats identical normalized strings as one node; document this limitation; disambiguation is a later brief
Co-occurrence noise (weak “relationships”)	<code>confidence</code> column = chunk-count weight; UI hides weight-1 edges by default
Indexing time growth	Deterministic inference only; measure on 1k-doc corpus before Phase 2
False relationships hurting trust	Every edge carries provenance (<code>doc_id</code> , <code>chunk_id</code>); UI links to source chunk

6. Effort Estimate (for planning)

- Phase 1: ~2 implementation briefs (backend+migration; frontend)
- Phase 2: 1 brief (chat integration + prompt update)
- No new infrastructure, no new services, no model downloads.

Bottom line: this is the highest intelligence-per-dollar upgrade available to Seekra right now — it monetizes work that Briefs #10/#13/#15 already paid for, without adding infrastructure or per-document LLM cost.